PropOS 前后端并发开发指南

> 版本: v2.0
>
> 日期: 2026-04-05
>
> 适用范围: Phase 1 全部模块（M1~M5）

一、核心原则：API Contract 先行

前后端并发开发的唯一前提是先签订 **API Contract**。Contract 是双方的接口约定，一旦确认即可独立开发，联调时只需将前端 Mock 数据切换为真实 API 调用，Pinia Store 与 UI 层零改动。

```
┌──────────────────────────────────────────────────────────┐
│                    每个模块启动前（1~2天）                  │
│                                                          │
│   后端输出 API Contract（JSON 字段名 + 类型 + 枚举值）     │
│          ↓                                               │
│   前端确认字段满足 UI 需求（可协商调整）                    │
│          ↓                                               │
│   Contract 冻结 → 并发开发开始                             │
└──────────────────────────────────────────────────────────┘
          ↓                                ↓
┌──────────────────┐            ┌──────────────────────┐
│    后端开发      │            │      前端开发        │
│  实现真实业务逻辑 │            │  基于 Mock 驱动 UI   │
│  数据库 Schema   │            │  Pinia Store + 页面 + 测试 │
└──────────────────┘            └──────────────────────┘
          └──────────────────────────┘
                          ↓
        联调（替换 Mock → 真实 HTTP）
```

二、API Contract 规范

2.1 Contract 定义格式

每个模块的 API Contract 以 Markdown 表格 + JSON 示例呈现，存放于 `docs/api/` 目录（如 `docs/api/m1_assets.md` ），不产生 **PDF，供开发期快速迭代。**

Contract 必须包含以下内容:

要素	说明
端点路径 + HTTP 方法	`GET /api/units`
请求参数（Query / Body）	字段名、类型、是否必填、默认值

响应 JSON 示例	完整的成功响应结构
枚举值列表	所有 string 枚举的合法值
错误 code 列表	该端点可能返回的 `error.code`

2.2 Contract 示例（M1 单元列表）

端点： `GET /api/units`

请求参数：

参数	类型	必填	默认值	说明
`page`	int	否	1	页码（从 1 开始）
`pageSize`	int	否	20	每页数量（最大 100）
`buildingId`	string(UUID)	否	—	按楼栋过滤
`propertyType`	string	否	—	`office` \
`status`	string	否	—	`leased` \

响应示例：

```
{
  "data": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "unitNo": "10A",
      "propertyType": "office",
      "grossArea": 120.5,
      "netArea": 105.0,
      "currentStatus": "leased",
      "floor": {
        "id": "uuid",
        "floorName": "10F",
        "floorNumber": 10
      },
      "building": {
        "id": "uuid",
        "name": "A座"
      },
      "currentContractId": "uuid-or-null",
      "daysUntilExpiry": 45,
      "svgHotzoneCoords": [{"x": 100, "y": 200}, {"x": 300, "y": 200}]
    }
  ],
  "meta": {
    "page": 1,
    "pageSize": 20,
    "total": 441
```

```
  }
}
```

枚举值：

字段	合法值
`propertyType`	`office` / `retail` / `apartment`
`currentStatus`	`leased` / `vacant` / `expiring_soon` / `non_leasable`

错误 **code**：

code	HTTP	场景
`BUILDING_NOT_FOUND`	404	`buildingId` 指定的楼栋不存在
`INVALID_REQUEST`	400	参数格式错误

2.3 Contract 变更规则

- Contract 冻结后，**后端不得单方面修改字段名或删除字段**；如需变更，必须与前端协商，前端确认后方可调整
- 新增字段（非 breaking change）无需协商，但需通知前端
- 枚举新增值视为非 breaking change；删除枚举值必须协商

三、后端开发工作流

3.1 阶段划分

```
阶段 A：输出 API Contract（同步前端）
阶段 B：实现路由骨架（Controller 空实现，可返回固定 Mock 数据供前端联调验证）
阶段 C：实现 Repository + Service 真实逻辑
阶段 D：集成测试 + 联调
```

3.2 路由骨架先行（阶段 B）

Contract 确认后，**优先建立可运行的路由骨架**，Controller 返回固定 JSON，前端可立即用真实 HTTP 替代 Mock：

```
// contracts/controllers/unit_controller.dart — 骨架阶段
Router get router {
  final r = Router();
  r.get('/api/units', _listUnits);
  r.get('/api/units/<id>', _getUnit);
  r.post('/api/units', _createUnit);
  return r;
}
```

```
Future<Response> _listUnits(Request request) async {
  // 阶段 B：返回固定 Mock，前端可先联调响应结构
  return Response.ok(jsonEncode({
    'data': [_mockUnit()],
    'meta': {'page': 1, 'pageSize': 20, 'total': 1},
  }), headers: {'content-type': 'application/json'});
}
```

3.3 分层实现顺序（阶段 C）

```
models/（freezed 数据类 + Command 对象）
  ↓
repositories/（SQL 查询，从简单 SELECT 开始）
  ↓
services/（业务规则，调用 packages/ 计算库）
  ↓
controllers/（替换骨架 Mock → 真实 Service 调用）
```

3.4 后端开发检查清单

每个端点实现完成前确认：

- [ ] RBAC 中间件已在路由上注册（架构约束 #1）
- [ ] 二房东相关查询已在 Repository 层加行级隔离过滤（约束 #2）
- [ ] 加密字段 API 响应已脱敏（约束 #3）
- [ ] 变更操作已触发审计日志写入（约束 #4）
- [ ] Controller 未直接返回 `Response`，业务异常通过 `AppException` 抛出
- [ ] Service 方法只接受强类型 Command 对象，无 `Map<String, dynamic>` 参数

四、前端开发工作流

4.1 三阶段开发模式

```
阶段 1：Contract 确认后 → 建立 TypeScript 类型定义（types/）+ API 函数（api/modules/）
阶段 2：Mock 数据 → 建立 Pinia Store + 页面 UI（本地 Mock 驱动）
阶段 3：联调 → Store 中的 API 调用切换到真实后端
```

4.2 阶段 1：类型定义与 API 函数（最先建立）

拿到 Contract JSON 示例后，立即定义 TypeScript 接口和 API 调用函数：

```
// app/src/types/unit.ts  或 admin/src/types/unit.ts
export interface Unit {
  id: string
  unitNo: string
  propertyType: 'office' | 'retail' | 'apartment'
  grossArea: number
```

```
  netArea?: number
  currentStatus: 'vacant' | 'leased' | 'non_leasable'
  floor: FloorSummary
  building: BuildingSummary
  currentContractId?: string
  daysUntilExpiry?: number
  svgHotzoneCoords?: HotzonePoint[]
}

// app/src/api/modules/unit.ts
import { apiGet } from '../client'
import { API_PATHS } from '@/constants/api_paths'
import type { PagedResponse } from '@/types/api'
import type { Unit } from '@/types/unit'

export function listUnits(params?: {
  page?: number
  pageSize?: number
  buildingId?: string
  propertyType?: string
  status?: string
}) {
  return apiGet<PagedResponse<Unit>>(API_PATHS.UNITS, { params })
}
```

4.3 阶段 2：Pinia Store + Mock 数据（驱动 UI 开发）

建立 Pinia Store，在后端未就绪时使用 Mock 数据：

```
// app/src/stores/unit.ts  或 admin/src/stores/unit.ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { listUnits as apiListUnits } from '@/api/modules/unit'
import type { Unit } from '@/types/unit'
import type { PageMeta } from '@/types/api'

// -- Mock 数据（联调后删除）--
const USE_MOCK = import.meta.env.VITE_USE_MOCK === 'true'
function mockUnit(i: number): Unit {
  return {
    id: `unit-${i}`,
    unitNo: `A-${1001 + i}`,
    propertyType: 'office',
    grossArea: 120.5,
    currentStatus: 'vacant',
    floor: { id: 'f1', name: '1F' },
    building: { id: 'b1', name: 'A栋' },
  }
}

export const useUnitStore = defineStore('unit', () => {
  const list = ref<Unit[]>([])
  const loading = ref(false)
```

```
  const error = ref<string | null>(null)
  const meta = ref<PageMeta>({ page: 1, pageSize: 20, total: 0 })

  async function fetchList(params?: Record<string, unknown>) {
    loading.value = true
    error.value = null
    try {
      if (USE_MOCK) {
        // Mock 模式：模拟延迟
        await new Promise(r => setTimeout(r, 300))
        list.value = Array.from({ length: 20 }, (_, i) => mockUnit(i))
        meta.value = { page: 1, pageSize: 20, total: 100 }
      } else {
        const res = await apiListUnits(params)
        list.value = res.data
        meta.value = res.meta
      }
    } catch (e) {
      error.value = e instanceof Error ? e.message : '操作失败，请重试'
    } finally {
      loading.value = false
    }
  }

  return { list, loading, error, meta, fetchList }
})
```

> **Mock 切换**：通过 `.env.development` 中设置 `VITE_USE_MOCK=true`，联调时改为 `false` 或删除该变量。

4.4 阶段 3：切换到真实 API（联调）

后端就绪后，只需将 `.env.development` 中 `VITE_USE_MOCK` 改为 `false`（或删除），Store 中的 API 调用会自动走真实后端。无需修改 Store 或页面代码。

4.5 前端开发检查清单

每个模块完成前确认：

- [ ] TypeScript 接口已定义在 `types/` 目录，与 Contract 一致
- [ ] API 函数封装在 `api/modules/` 中，使用 `api_paths` 常量
- [ ] Pinia Store 使用 `defineStore(id, setup)` 风格，state 含 `list` / `item` / `loading` / `error` / `meta`
- [ ] Store 通过 `api/client` 调用后端，未直接使用 `fetch` / `axios`
- [ ] Page / Component 只访问 Store 的 state / action，不内联 HTTP 请求
- [ ] 所有颜色通过 `Theme.of(context).colorScheme.*`（Flutter）或 Element Plus `type` 属性 / CSS 变量（admin）取值，无硬编码
- [ ] 状态色使用语义映射（`success` / `warning` / `danger` / `info`）
- [ ] 常量已归入对应的常量文件，无魔法数字

五、联调流程

5.1 联调触发条件

满足以下两条即可开始联调，无需等后端全部完成：

1. 后端目标端点已部署到开发环境（哪怕仍返回 Mock 数据）
2. 前端 HTTP Repository 实现已完成

5.2 联调步骤

```
Step 1: 后端部署到本地或开发服务器
        export DATABASE_URL=... JWT_SECRET=...  （必填环境变量）
        dart run bin/server.dart

Step 2: 前端切换到真实模式
        # .env.development 中设置 VITE_USE_MOCK=false
        npm run dev

Step 3: 逐端点验证
        ✓ 正常响应结构与 Contract 一致
        ✓ 分页参数有效
        ✓ 错误场景返回正确 error.code（前端 Store 按 code 处理）
        ✓ 加密字段已脱敏（仅显示后4位）
        ✓ 日期字段格式正确（ISO 8601 UTC）

Step 4: 发现偏差时
    字段名不一致 → 后端修复（Contract 优先）
    字段缺失 → 协商后更新 Contract 再双端修复
    业务逻辑分歧 → 后端 Service 层修正
```

5.3 开发环境配置建议

角色	本地配置
后端开发	`.env` 文件注入环境变量，`dart run bin/server.dart` 启动
前端开发（Mock 阶段）	`.env.development` 中 `VITE_USE_MOCK=true`，`npm run dev` 启动
前端开发（联调阶段）	`.env.development` 中 `VITE_USE_MOCK=false` + `VITE_API_BASE_URL=http://localhost:8080`，`npm run dev`

六、分模块并发开发计划（Phase 1）

建议按以下顺序启动，每个模块均遵循"Contract 先行 → 双端并发 → 联调"流程：

模块	Contract 依赖	推荐启动顺序	说明
M1 资产与空间	无	**第1批**	其他模块依赖 Unit/Building，须先完成
Auth（登录鉴权）	无	**第1批**	Pinia 状态管理 + JWT 存储，须先完成
M2 租务合同	依赖 M1 Unit	第2批	Contract 实体依赖 Unit 已存在
M4 工单系统	依赖 M1 Unit	第2批	WorkOrder 定位依赖 Unit/Floor
M3 财务 NOI	依赖 M2 Contract	第3批	Invoice 依赖 Contract
M5 二房东穿透	依赖 M1 + M2	第3批	SubLease 依赖 Contract + Unit

> 并发原则：同批次内的模块前后端可完全并发；跨批次模块等上一批次的 **TypeScript 类型定义**（`types/` + API 函数）完成后即可启动，无需等待真实 API 完成。

七、常见问题

Q：后端字段返回 `snake_case`，前端 TypeScript 接口用 `camelCase`，如何处理？

后端 API 统一返回 `camelCase`（在 Controller 序列化时转换），不使用 `snake_case`。前端 TypeScript 接口直接对应即可。

Q：前端 Mock 数据与真实 API 响应结构不一致怎么办？

以 Contract JSON 示例为准。Mock 数据必须严格按照 Contract 结构构造，发现不一致时优先修复 Mock（而不是等联调）。

Q：后端某端点比预期晚完成，前端如何处理？

前端保持 Mock 模式继续开发，该端点的 Store 中 API 调用保持 Mock 分支即可，等后端就绪后切换 `VITE_USE_MOCK=false`。

Q：多个开发者同时修改同一模块如何避免冲突？

- 后端：按 `models → repositories → services → controllers` 顺序分工，单人负责一个子模块
- 前端：按 `types → api → store → pages/components` 分工，`types/` 和 `api/` 层优先由一人完成，其余层可并行

Q：`env` 环境变量本地如何管理？

后端根目录创建 `.env`（已加入 `.gitignore`），格式：

```
DATABASE_URL=postgres://user:pwd@localhost:5432/propos
JWT_SECRET=local-dev-secret-min-32-chars-long
JWT_EXPIRES_IN_HOURS=24
FILE_STORAGE_PATH=/tmp/propos_uploads
ENCRYPTION_KEY=0000000000000000000000000000000000000000000000000000000000000000
APP_PORT=8080
```

项目根目录提供 `.env.example` （含所有变量名，值填「示例占位符」），新成员克隆后复制 `.env.example` 为 `.env` 再填入真实值。